**Assignment 1 Report: Job Search Tracker**

Alexandra María Molins Gutiérrez

Software Development and DevOps

## 1. Introduction

Job Search Tracker is a single-process Flask web application that can track job applications through a pipeline and tells you which skills you must learn next based on what your active applications are asking for.

When you start to apply for jobs you have to apply to at least 20-30 hobs at the same time, and it is easy to lose track of the stages you are in throughout your applications, and you also spend time studying for skills that are probably not required.

The primary user of this web app are students who are looking for jobs, or graduate students who are applying to jobs. A possible stakeholder could be Universities as this tool can be offered to them so they can help students keep their applications organized.

**Feature domains:**

1. **Application pipeline:** The applications move across applied → phone screen →technical assessment → interview → final round → offer. Applications that haven't been moved or touched for more than 14 days are highlighted, and for each way of applying it shows the number of applications that made it to the next stage to see which one works best.
2. **Skill Gap Tracker:** There is a list of the skills you have with a level of 1-5 and you can mark the skills the application requires. The app ranks missing skills by how many active applications want them and gives a priority to each skill needed by doing applications needing that skill x (5- the level you have)

The app runs as one process using python with one SQLite file and one dependency manifest.

## 2. SDLC model and justification

The SDLC model I used was an Agile model which is defined as an iterative, incremental delivery in short cycles. Each day I worked in the project was one cycle in which I had to go through requirements, design, development and testing.

I decided to choose agile instead of waterfall because waterfall is linear and rigid and its weakness is that design problems are found later when testing. Agile was great and well suited for this assignment as we were asked to work throughout 6 days instead of only one delivery at the end and this model allows us to work best by testing and fixing in small increments.

SDLC phases:

Each day went through every SDLC phase. Planning: a day-by-day plan with 2 commits per day for 6 days. Requirements analysis: functional and non-functional requirements from the

approved project description. Design: blueprints and layers at a high level. Development: flask code one increment per day. Testing: 154 unit, integration and page tests that have 100% coverage. Deployment: runs locally and inside a single Docker container built from the template provided. Maintenance: restyling and added an accepted state.

**SMART goals**

From the beginning I set 4 SMART goals: both domains reading and writing SQLite by October 6th, then by October 7th at least 70% of test coverage for pipeline and skills and I exceeded with 100% coverage and 154 tests, I had at least 12 commits on 6 or more days, I included the 5 ADR entries on at least three dates. Furthermore, I followed the model correctly but had to adapt some things such as replacing the use of shadcn/ui since it needed JavaScript and a second dependency file, so it was replaced by plain CSS, after testing I added an accepted state as I hadn't considered it before. The place in which I don't follow a continuous process was with the AI-USAGE.md which I did at the end and committed on the last day.

## 3. Architecture overview

The app runs as a single Python process that is started with python app.py. The server will build every page using Jinja templates and send it back to the browser as an HTML. Every action is an HTML form that sends a POST request to the server, and the server answers with a redirect to a fresh page.

When the app starts, the create_app() function inside app.py it reads the settings from environment variables. Then the data folder is created and all the database tables, so it doesn't have to be manual, and registers the two parts of the app as Flask blueprints, the application pipeline under application and the skill gap tracker under skills. Finally, the server reads it from port 8000 which is the default and listens on 0.0.0.0 as required.

Both domains are organized the same way using three layers: logic.py which holds rules as python fucntions, repository.py holds all the SQL reads and writes to the databse, and routes.py connects the two to the web.

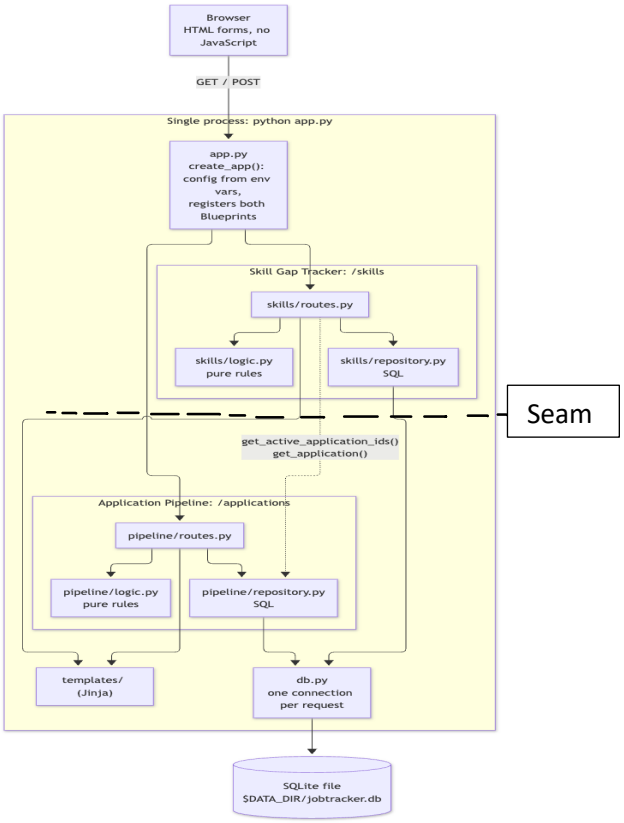



*Figure 1: Architecture overview. The dashed line marks the seam between the two domains. The only call that crosses it is the skills domain calling get_active_application_ids() and get_application() from the pipeline repository.*

## 4. Database model/schema diagram

All data lives in one SQLite file, in five different tables created by schema.sql on every startup, but since every statement is Create table if not exists, it creates only the missing tables and never deletes or changes existing data.

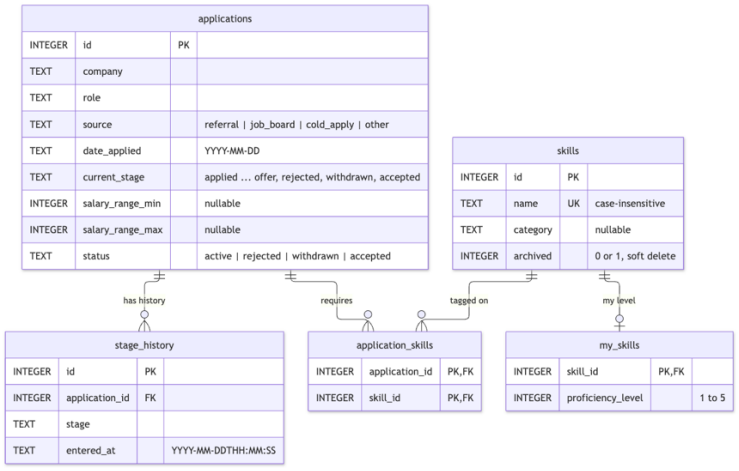



*Figure 2*:

## 5. Setup and README summary

The README in the repository has all the instructions needed.

git clone https://github.com/alemol123/JobSearchTracker.git
cd JobSearchTracker
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python app.py  # then open http://localhost:8000

**Run with Docker:**
docker build -t jobtracker .
docker run -p 8000:8000 -v jobtracker-data:/data jobtracker
With uv installed, uv run --no-project --with-requirements requirements.txt python app.py does the same without creating a virtual environment by hand.

**Configuration:**

PORT (default 8000), DATA_DIR (default ./data, /data in Docker), STALE_DAYS (default 14), SECRET_KEY (default dev-only-change-me). The database file is $DATA_DIR/jobtracker.db. There is no seed data and no committed .db file.

**Tests:**

pytest --cov=pipeline --cov=skills --cov-report=term-missing → 154 passed, 100% coverage on pipeline and skills.

**Container evidence:**

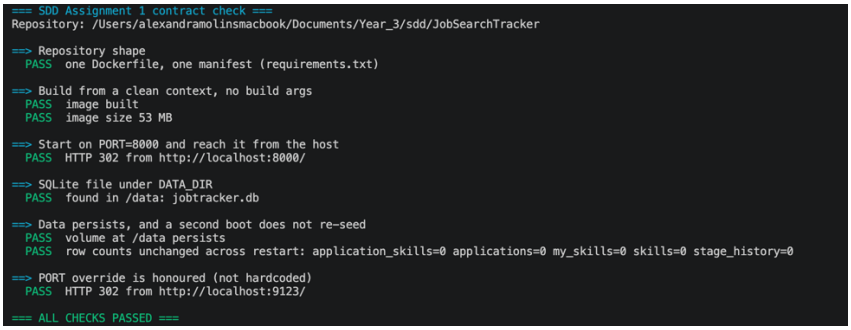



*Figure 3: Output of the provided run.sh checker*

## 6. AI disclosure

I acknowledge the use of Claude Code to plan a daily schedule and generate code for the application and explain concepts I asked. The prompts used include “Guide me step by step” ,“Give me a 6 day plan with 12 commits in total”, “Would you recommend using shadcn for the design?”. The output of these prompts was used to build the flask application, pytest suite, diagrams. Every interaction was recorded in the AI_USAGE.md.